

Al-Furqan Engine — Sprints 3-5

Documentation

Complete Technical Reference

Project: Al-Furqan (الفرقان) — Axiom-Anchored Neuro-Symbolic Reasoning Engine
Version: 1.0
Date: March 21, 2026
Scope: Engine Refactor (Sprint 3A), KB Infrastructure (3B-3E), Gate Decomposition (4A), Chains + Scorer (4B), Z3 Symbolic Verification (4C), Human Feedback (4D), Orchestrator (5A)

Table of Contents

- 1. [Sprint Overview](#)
- 2. [Directory Structure](#)
- 3. [Sprint 3A: Engine Refactor](#)
- 4. [Sprint 3B: Embedding Infrastructure](#)
- 5. [Sprint 3C: Knowledge Base Collections](#)
- 6. [Sprint 3D: Knowledge Graph](#)
- 7. [Sprint 3E: Integration Tests](#)
- 8. [Sprint 4A: Gate Decomposition](#)
- 9. [Sprint 4B: Chains + Scorer](#)
- 10. [Sprint 4C: Z3 Symbolic Verification](#)
- 11. [Sprint 4D: Human Feedback](#)
- 12. [Sprint 5A: Orchestrator](#)
- 13. [End-to-End Data Flow](#)
- 14. [Test Coverage Summary](#)

1. Sprint Overview

Sprint	Task	Status	Key Deliverable
3A	Engine Refactor + Model Metadata	✓ Complete	engine/ module with axioms, models, pipeline, prompts
3B	Embedding Infrastructure	✓ Complete	kb/embeddings.py — CamelBERT + MiniLM
3C	KB Collections	✓ Complete	Quran (6,236), Hadith (38,016+), Fiqh (50+)
3D	Knowledge Graph	✓ Complete	Graph schema, store, traversal
3E	Integration Tests	✓ Complete	KB integration test suite
4A	Gate Decomposition	✓ Complete	4 independent gate modules

4B	Chains + Scorer	✓ Complete	Chain executor + deterministic scorer
4C	Z3 Symbolic Verification	✓ Complete	Formal axiom encoding, predicate extractor, verifier
4D	Human Feedback	✓ Complete	Feedback store with ChromaDB + SQLite
5A	Orchestrator	✓ Complete	Full pipeline orchestration with security

Test count evolution: 205 (Sprint 2) → **647 total** (560 engine + 87 skills)

2. Directory Structure

```

src/al_furqan/
├─ engine/                                # Layer 1: Furqan Engine (الفرقان)
│   ├── __init__.py
│   ├── axioms.py                        # Immutable axioms + SHA-256 hash
│   ├── models.py                        # Verdict, GateScore, DualPerspectiveVerdict,
etc.
│   ├── pipeline.py                      # Scan → Mirror → Verdict → Self-Correction
│   ├── prompts.py                      # All prompt templates + input sanitization
│   ├── gates/                           # Individual gate implementations
│   │   ├── __init__.py
│   │   ├── base.py                     # Abstract Gate base class
│   │   ├── source_integrity.py          # Gate 1: Source Integrity (المصدر)
│   │   ├── structural_consistency.py    # Gate 2: Structural Consistency (البنية)
│   │   ├── mediation_zeroing.py         # Gate 3: Mediation Zeroing (الوساطة)
│   │   └── origin_aware.py              # Gate 4: Origin Aware (الأصل) – BINARY
│   ├── chains/                          # Guided reasoning chains
│   │   ├── __init__.py
│   │   ├── definitions.py              # Chain questions per gate
│   │   ├── executor.py                 # LLM-driven fact extraction
│   │   └── scorer.py                   # Deterministic Python scoring
│   ├── symbolic/                        # Z3 formal verification
│   │   ├── __init__.py
│   │   ├── formal_axioms.py            # Z3-encoded axioms (3 axioms + 2 proofs)
│   │   ├── predicate_extractor.py       # Maps evaluation data → Z3 predicates
│   │   └── verifier.py                 # SAT/UNSAT/UNKNOWN verification
│   └── security/                         # Security hardening (Sprint 6)
│       ├── __init__.py
│       ├── integrity.py                 # Axiom hash verification
│       ├── prompt_guard.py               # Injection detection
│       ├── output_validator.py           # Output structure validation
│       ├── adapter_sandbox.py            # Domain adapter sandboxing
│       └── audit.py                     # Privacy-preserving audit logs
├─ kb/                                   # Layer 2: Knowledge Base (المعرفة)
└─ __init__.py

```

```

├── embeddings.py                # EmbeddingModel – CamelBERT/MiniLM
├── retriever.py                 # UnifiedRetriever – cross-collection search
├── knowledge_linker.py          # Graph-enhanced reasoning chains
├── collections/                 # Data collections
│   ├── __init__.py
│   ├── quran.py                # 6,236 verses + tafsir
│   ├── hadith.py               # 38,016+ hadith with grading
│   └── fiqh.py                 # 50+ core fiqh rules
├── graph/                       # Knowledge graph
│   ├── __init__.py
│   ├── schema.py              # Node/Edge type definitions
│   ├── store.py               # Graph database operations
│   └── traversal.py           # Multi-hop graph traversal
├── ingestion/                  # Data ingestion pipelines
│   ├── __init__.py
│   ├── ingest_quran.py
│   ├── ingest_hadith.py
│   └── ingest_fiqh.py
├── store/                      # Layer 3: Storage (التخزين)
│   ├── __init__.py
│   ├── verdict_store.py       # ChromaDB verdict persistence
│   ├── feedback_store.py     # Human feedback storage
│   └── (pattern_store – planned)
├── api/                        # Layer 4: Orchestration & API
│   ├── __init__.py
│   ├── app.py                 # FastAPI application
│   ├── orchestrator.py       # Central pipeline orchestrator
│   ├── schemas.py            # Pydantic request/response schemas
│   ├── converters.py         # Data conversion utilities
│   ├── dependencies.py       # FastAPI dependency injection
│   └── routers/
│       ├── __init__.py
│       ├── evaluate.py       # POST /evaluate, /evaluate-grounded
│       ├── verdicts.py      # GET/DELETE /verdicts
│       ├── criterion.py     # Criterion info endpoint
│       ├── review.py        # Human review endpoints
│       └── stats.py         # System statistics
├── auth/                      # Authentication (Sprint 2)
│   ├── __init__.py
│   ├── key_manager.py
│   ├── middleware.py
│   ├── security.py
│   ├── rate_limiter.py
│   ├── models.py
│   ├── errors.py
│   └── cli.py
├── core/                      # Legacy engine (Sprint 1-2, kept for
compatibility)

```

```
├── __init__.py
├── reasoning_engine.py           # Original monolithic engine
├── cot.py                       # Chain of Thought v1
├── cot_engine.py
├── cot_prompts.py
├── providers/                   # LLM Provider Layer
│   ├── __init__.py
│   └── llm_layer.py            # Multi-provider LLM abstraction
├── review/                     # Human Review
│   ├── __init__.py
│   └── human_review.py
├── cli.py                      # CLI entry point
└── config.py                   # YAML-based configuration
```

3. Sprint 3A: Engine Refactor

3A.1 — Axioms Module (engine/axioms.py)

Extracted all immutable axiom content from the monolithic `reasoning_engine.py` into a standalone module with versioning and integrity hashing.

Key constants:

Constant	Description
AXIOM_VERSION	"1.0.0" — semantic version for the axiom set
FRAMEWORK_PREAMBLE	System identity prompt for The Criterion
AXIOMS	Transcendence Necessity Proof + Final Court Necessity Proof + Core Axioms
GATE_DEFINITIONS	Tri-Axial Survival Gates (4 gates)
SCORING_RULES	Deterministic scoring formula
AXIOM_HASH	SHA-256 hash of all content combined

Integrity verification:

```
from al_furqan.engine.axioms import AXIOM_HASH, _compute_axiom_hash

# At startup and before every evaluation:
assert _compute_axiom_hash() == AXIOM_HASH, "Axiom tampering detected!"
```

3A.2 — Data Models (engine/models.py)

All data structures extracted into a standalone module:

Class	Type	Purpose
-------	------	---------

SystemType	Enum	economic, social, spiritual, political, legal, technological, environmental, mixed
GateResult	Enum	Survive / Fail
GateScore	Dataclass	name, score (0-100), result, reasoning
Verdict	Dataclass	Full evaluation result with model metadata
DualPerspectiveVerdict	Dataclass	System verdict + assumptions verdict
InformationalResponse	Dataclass	Response for non-evaluative questions

Verdict model metadata fields (Sprint 3A.4):

```
@dataclass
class Verdict:
    # ... core fields ...
    model_provider: Optional[str] = None      # e.g., "anthropic"
    model_name: Optional[str] = None          # e.g., "claude-sonnet-4-20250514"
    model_temperature: Optional[float] = None # e.g., 0.0
    raw_scan_response: Optional[str] = None   # Full LLM response for scan
    raw_mirror_response: Optional[str] = None
    raw_verdict_response: Optional[str] = None
```

Serialization: `to_dict()` / `from_dict()` with full round-trip support.

3A.3 — Prompt Templates (`engine/prompts.py`)

All prompt builders extracted with input sanitization:

Function	Phase	Purpose
<code>sanitize_input(text)</code>	Pre-processing	Removes injection patterns, enforces 5000 char limit
<code>build_intent_detection_prompt(question)</code>	Phase 0	Classifies: <code>system_evaluation</code> / <code>claim_judgment</code> / <code>informational</code>
<code>build_informational_prompt(question)</code>	Phase 0b	Direct factual answer (no gates)
<code>build_scan_prompt(question, context)</code>	Phase 1	Identify system type, effects, friction points
<code>build_mirror_prompt(question, scan_result)</code>	Phase 2	Evaluate through all 4 gates
<code>build_verdict_prompt(question, scan, mirror)</code>	Phase 3	Consequences + final judgment
<code>build_correction_prompt(question, verdict, pass_number)</code>	Phase 4	Self-correction for contradictions

Input sanitization patterns:

```

INJECTION_PATTERNS = [
    r'(?i)ignore\s+(all\s+)?(previous|above|prior)\s+(instructions?|prompts?|rules?)',
    r'(?i)you\s+are\s+now\s+',
    r'(?i)system\s*:\s*',
    r'(?i)\[INST\]',
    r'(?i)<\|im_start\|>',
    # ... 8 patterns total
]

```

3A.5 — Pipeline Module (engine/pipeline.py)

The `EvaluationPipeline` class implements the core reasoning flow:

```

Input Question
  ↓
Phase 0: Intent Detection
  ├── informational → InformationalResponse (skip gates)
  └── evaluative → Continue
  ↓
Phase 1: Scan (identify system type, effects, friction)
  ↓
Phase 2: Mirror (evaluate through 4 gates)
  ↓
Phase 3: Verdict (consequences + judgment)
  ↓
Phase 4: Self-Correction Loop (up to 5 passes)
  ↓
Verdict Object

```

Key design: LLM decoupling

```

class EvaluationPipeline:
    def __init__(self, llm_call: Callable[[str], str]):
        """Accepts ANY callable that takes a prompt and returns text."""
        self.llm_call = llm_call

```

This means any LLM (Claude, Qwen, Ollama, or even a mock) can be plugged in.

JSON parsing strategy (3 levels of fallback):

1. Try direct `json.loads()`
2. Extract from markdown code fences (````json ... ````)
3. Find outermost `{...}` braces + repair common JSON errors

4. Sprint 3B: Embedding Infrastructure

File: `kb/embeddings.py`

Class: `EmbeddingModel`

Abstraction over embedding models with automatic fallback.

Model Key	HuggingFace Path	Dimension	Use Case
camelbert	CAMEL-Lab/bert-base-arabic-camelbert-ca	768	Arabic-optimized (production)
minilm	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	384	Lightweight multilingual (default)

Key methods:

```
class EmbeddingModel:
    def embed(self, texts: list[str]) -> list[list[float]]
    def embed_query(self, query: str) -> list[float]
    def similarity(self, text_a: str, text_b: str) -> float

    @property
    def dimension(self) -> int
    @property
    def model_name(self) -> str
```

Fallback behavior: If CamelBERT fails to load, automatically falls back to MiniLM with a warning.

All embeddings are L2-normalized for cosine similarity compatibility.

5. Sprint 3C: Knowledge Base Collections

5.1 Quran Collection (kb/collections/quran.py)

Field	Type	Description
surah	int	Surah number (1-114)
ayah	int	Ayah number
text_ar	str	Arabic text
text_en	str	English translation
surah_name_en	str	English surah name
juz	int	Juz number (1-30)

QuranVerse dataclass with full metadata. **QuranCollection** supports:

- search(query, limit) — semantic + keyword search
- get_verse(surah, ayah) — direct lookup
- get_context(surah, ayah, window) — surrounding verses

5.2 Hadith Collection (kb/collections/hadith.py)

Field	Type	Description
collection_name	str	Bukhari, Muslim, etc. (10 collections)
number	int	Hadith number
text_ar	str	Arabic text
text_en	str	English translation
narrator	str	Chain of narration
grading	str	sahih / hasan / daif

HadithCollection supports grading-filtered search.

5.3 Fiqh Collection (kb/collections/fiqh.py)

50+ core fiqh rules including the Five Major Rules (القواعد الخمس الكبرى):

1. الأمور بمقاصدها — Matters by their intentions
2. اليقين لا يزول بالشك — Certainty not removed by doubt
3. المشقة تجلب التيسير — Hardship brings ease
4. الضرر يُزال — Harm must be eliminated
5. العادة محكمة — Custom is authoritative

Each rule mapped to supporting Quran/Hadith evidence.

5.4 Unified Retriever (kb/retriever.py)

Class: `UnifiedRetriever`

Searches across all collections, merges, deduplicates, and formats:

```
class UnifiedRetriever:
    def retrieve(self, query: str, config: RetrievalConfig = None) ->
KnowledgeContext
```

KnowledgeContext contains:

- `results: list[RetrievalResult]` — individual results with source, Arabic/English content, reference
- `formatted_text: str` — formatted text block grouped by source type (Quran Evidence / Hadith Evidence / Fiqh Rules)
- `query: str` — original query
- `sources_searched: list[Source]` — which collections were searched

Source enum: `QURAN` , `HADITH` , `FIQH`

RetrievalConfig:

- `sources: list[Source]` — which collections to search (default: all)
- `limit_per_source: int` — max results per collection (default: 3)
- `hadith_grading_filter: Optional[str]` — filter by grading

6. Sprint 3D: Knowledge Graph

Schema (`kb/graph/schema.py`)

Node Types:

Node	Key Fields
Ayah	surah, ayah, text_ar, text_en, topics[]
Hadith	collection, number, text_ar, grading, narrator
FiqhRule	text_ar, text_en, category
Scholar	name
Topic	name_ar, name_en
Maqsad	name_ar, name_en (Maqasid al-Shariah)

Edge Types:

Edge	From → To
INTERPRETED_BY	Ayah → Hadith
EXPLAINED_BY	Ayah/Hadith → Lesson
ESTABLISHES	Ayah/Hadith → FiqhRule
TAGGED_WITH	Ayah/Hadith → Topic
SERVES_MAQSAD	FiqhRule → Maqsad
RELATES_TO	Ayah → Ayah

Graph Store (`kb/graph/store.py`)

CRUD + traversal operations for the knowledge graph.

Graph Traversal (`kb/graph/traversal.py`)

Multi-hop traversal queries for building scholarly reasoning chains.

Knowledge Linker (`kb/knowledge_linker.py`)

Builds reasoning chains from graph traversal:

- Start from retrieved sources
- Expand via graph relationships (1-3 hops)
- Build chains connecting verse → hadith → fiqh rule → maqsad

7. Sprint 3E: Integration Tests

KB integration tests in:

- tests/test_embeddings.py — 17 tests
- tests/test_quran_collection.py — 14 tests
- tests/test_hadith_collection.py — 13 tests
- tests/test_fiqh_collection.py — 18 tests
- tests/test_retriever.py — 15 tests
- tests/test_graph_store.py — 30 tests
- tests/test_graph_integration.py — 12 tests
- tests/test_knowledge_linker.py — 12 tests
- tests/test_kb_integration.py — 6 tests

Total Sprint 3 tests: 137

8. Sprint 4A: Gate Decomposition

Abstract Base (engine/gates/base.py)

```
class Gate(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...

    @property
    @abstractmethod
    def description(self) -> str: ...

    @abstractmethod
    def evaluate(self, chain_results: dict) -> GateScore:
        """PURE PYTHON – no LLM calls. Takes extracted facts, returns score."""

    @abstractmethod
    def get_chain_questions(self) -> list[str]:
        """Questions for LLM fact extraction."""
```

Gate 1: Source Integrity (المصدر)

File: engine/gates/source_integrity.py

Evaluates data fidelity and source origin.

Scoring formula:

```
base_score = SOURCE_TYPE_SCORES[source_type]
              divine=100, prophetic=80, scholarly=60, human_theory=40, unknown=20

score = base_score × (1.0 if verifiable, 0.5 if not)

if contradicts_primary:
    score -= 40

score = clamp(score, 0, 100)
result = SURVIVE if score >= 50 else FAIL
```

Chain questions (5):

1. What is the primary source of this claim? (divine/prophetic/scholarly/human_theory/unknown)
2. Is the source verifiable through chains of transmission, evidence, or proof?
3. Classify the source type exactly
4. Does it contradict Quran or authenticated Sunnah?
5. Is there reduction or reinterpretation for human convenience?

Gate 2: Structural Consistency (البنية)

File: engine/gates/structural_consistency.py

Evaluates causal mapping and logical coherence.

Scoring formula:

```
base_score = CONTRADICTION_SCORES[level]
              no_contradictions=90, minor_inconsistencies=60, major_contradictions=30

if causal_chain_intact: score += 10
if logical_gaps: score -= 20

Survive threshold: 50
```

Gate 3: Mediation Zeroing (الوساطة)

File: engine/gates/mediation_zeroing.py

Human noise audit — does it treat humans as observers, not masters of truth?

Scoring formula:

```
base_score = FOUNDATION_SCORES[type]
              non_human_foundation=90, mixed_foundation=50, pure_human_preference=20

if removes_bias: score += 10
if cultural_relativism: score -= 30

Survive threshold: 50
```

Gate 4: Origin Aware (الأصل)

File: engine/gates/origin_aware.py

BINARY gate — no numeric range:

- Acknowledges transcendent origin → **Survive (100)**
- Denies/ignores transcendent source → **Fail (0)**

Gate tests:

- tests/test_gate_source_integrity.py — 12 tests
 - tests/test_gate_structural_consistency.py — 8 tests
 - tests/test_gate_mediation_zeroing.py — 8 tests
 - tests/test_gate_origin_aware.py — 6 tests
-

9. Sprint 4B: Chains + Scorer

Chain Definitions (engine/chains/definitions.py)

Each gate has 3-5 guided questions stored in dictionaries:

```
GATE_CHAINS = {
    "Source Integrity (المصدر)": SOURCE_INTEGRITY_CHAIN,      # 5 questions
    "Structural Consistency (البنية)": STRUCTURAL_CONSISTENCY_CHAIN, # 4 questions
    "Mediation Zeroing (الوساطة)": MEDIATION_ZEROING_CHAIN,  # 4 questions
    "Origin Aware (الأصل)": ORIGIN_AWARE_CHAIN,               # 3 questions
}
```

Total: **16 chain questions** across all gates.

Chain Executor (engine/chains/executor.py)

```
class ChainExecutor:
    def __init__(self, llm_fn: Callable[[str], str]):
        """The ONLY place LLM calls happen for fact extraction."""

    def execute_chain(self, question: str, gate: Gate, context: str = "") -> dict:
        """Execute guided chain questions, accumulating context."""

    def execute_all_gates(self, question: str, gates: list[Gate], context: str = "")
    -> dict:
        """Execute chains for all gates, returns {gate_name: extractions}."""
```

Key principle: The LLM extracts facts. It does NOT score or judge. Each chain question builds on accumulated context from previous answers.

Deterministic Scorer (engine/chains/scorer.py)

```
class DeterministicScorer:
    def score_gate(self, gate: Gate, extractions: dict) -> GateScore:
        """Pure Python – NO LLM involvement."""

    def score_all_gates(self, gates: list[Gate], all_extractions: dict) ->
    list[GateScore]:
        """Score all gates deterministically."""

    def compute_total_score(self, gate_scores: list[GateScore]) -> int:
        """Average of all gate scores, clamped to [0, 100]."""
```

Determinism guarantee: Same extracted facts → same score, 100% of the time.

Tests:

- tests/test_chain_executor.py — 6 tests
- tests/test_deterministic_scorer.py — 9 tests

10. Sprint 4C: Z3 Symbolic Verification

Formal Axioms (engine/symbolic/formal_axioms.py)

All axioms encoded as Z3 first-order logic formulas:

Sorts:

- Entity — anything that exists (actions, systems, ideas)
- Framework — an evaluative system being tested

Axiom 1 — Design: $\forall x: \text{Exists}(x) \rightarrow \text{HasPurpose}(x)$

Axiom 2 — Network: $\forall x: \text{Exists}(x) \rightarrow \text{HasCausalNetwork}(x)$

Axiom 3 — Alignment: $\forall x: \text{Exists}(x) \rightarrow (\text{Aligned}(x) \leftrightarrow \text{Functional}(x))$

Proof 1 — Transcendence Necessity: $\forall f: \text{IsContingent}(f) \rightarrow (\neg \text{CanSelfGround}(f) \wedge \text{HasTranscendentSource}(f))$

Proof 2 — Final Court Necessity: $\forall f: (\text{HasMoralDebts}(f) \wedge \neg \text{HumanJusticeSufficient}(f)) \rightarrow \text{RequiresFinalCourt}(f)$

Gate-related predicates:

- HasVerifiedSource(Entity) — Gate 1
- IsInternallyConsistent(Entity) — Gate 2
- FreeFromHumanMediation(Entity) — Gate 3
- AcknowledgesTranscendence(Entity) — Gate 4
- PreservesNatural(Entity) — cross-gate

Sanity check function:

```
def check_axioms_satisfiable() -> bool:
    """Verify the axioms themselves don't contradict each other."""
    s = Solver()
    for ax in ALL_AXIOMS:
        s.add(ax)
    return s.check() == sat # Must return True
```

Predicate Extractor (engine/symbolic/predicate_extractor.py)

Maps structured evaluation results to Z3 boolean assertions:

Chain Result Key	Z3 Predicate	Inversion
source_type == "divine"	HasVerifiedSource(e)	No
has_contradictions == False	IsInternallyConsistent(e)	Yes (inverted)
relies_on_human_preference == False	FreeFromHumanMediation(e)	Yes (inverted)
acknowledges_transcendence == True	AcknowledgesTranscendence(e)	No
is_contingent == True	IsContingent(fw)	No

has_transcendent_source == True	HasTranscendentSource(fw)	No
---------------------------------	---------------------------	----

Symbolic Verifier (engine/symbolic/verifier.py)

```
class SymbolicVerifier:
    def __init__(self, timeout_ms: int = 10000):
        """10-second default timeout for Z3 solver."""

    def verify(self, predicates: list) -> VerificationResult:
        """Check predicates against axiom system. Returns SAT/UNSAT/UNKNOWN."""

    def verify_gate_consistency(self, gate_results: dict) -> VerificationResult:
        """Extract predicates from gate results and verify."""

    def verify_verdict(self, verdict_data: dict) -> VerificationResult:
        """Full pipeline: extract + verify."""

    def verify_per_gate(self, verdict_data: dict) -> dict[str, VerificationResult]:
        """Per-gate independent verification – 4 separate Z3 checks."""
```

VerificationResult:

```
@dataclass
class VerificationResult:
    consistent: Optional[bool] # True=SAT, False=UNSAT, None=UNKNOWN
    proof: str # Human-readable explanation
    contradictions: list # Details when UNSAT
    verification_time_ms: float
```

Per-gate verification (verify_per_gate):

Each gate gets its own independent Z3 check with only the predicates relevant to that gate:

- **Gate 1 (Source Integrity):** Checks if human_theory claiming self-grounding → contradiction
- **Gate 2 (Structural Consistency):** Checks if contradictions + claiming functional → violates Alignment axiom
- **Gate 3 (Mediation Zeroing):** Checks if pure_human_preference + contingent + no transcendent source → violates Transcendence axiom
- **Gate 4 (Origin Aware):** Checks if contingent + denies transcendence → direct axiom violation

Tests:

- tests/test_z3_axioms.py — 19 tests
- tests/test_symbolic_verifier.py — 14 tests
- tests/test_predicate_extractor.py — 12 tests

11. Sprint 4D: Human Feedback

Feedback Store (store/feedback_store.py)

Stores human corrections and ratings linked to verdicts:

```
@dataclass
class HumanFeedback:
    verdict_id: str
    reviewer: str
    rating: str          # "correct", "partially_correct", "incorrect"
    gate_corrections: dict
    notes: str
    timestamp: float
```

Tests: tests/test_feedback_store.py — 15 tests

12. Sprint 5A: Orchestrator

File: api/orchestrator.py

The Orchestrator is the **only component** that knows about all layers:

```
class Orchestrator:
    def __init__(
        self,
        engine_pipeline,          # EvaluationPipeline
        kb_retriever=None,        # UnifiedRetriever
        graph_store=None,        # GraphStore
        knowledge_linker=None,    # KnowledgeLinker
        verdict_store=None,       # VerdictStore
        feedback_store=None,      # FeedbackStore
        symbolic_verifier=None,   # SymbolicVerifier
        llm_fn=None,              # LLM callable
    ):
```

Security components wired in:

- IntegrityVerifier — axiom hash check before every evaluation
- PromptGuard — injection detection on input
- OutputValidator — verdict structure validation
- AuditLogger — privacy-preserving audit trail

Evaluation Pipeline Flow

1. Generate evaluation ID
2. Verify axiom integrity (IntegrityVerifier.verify_or_die())
3. Scan for prompt injection (PromptGuard)
4. KB retrieval + graph expansion (if use_kb=True)
5. Gate evaluation (EvaluationPipeline.evaluate())
6. Z3 verification (if use_z3=True)
7. Generate user-facing response (LLM as tongue)
8. Validate output (OutputValidator)
9. Store verdict (VerdictStore)

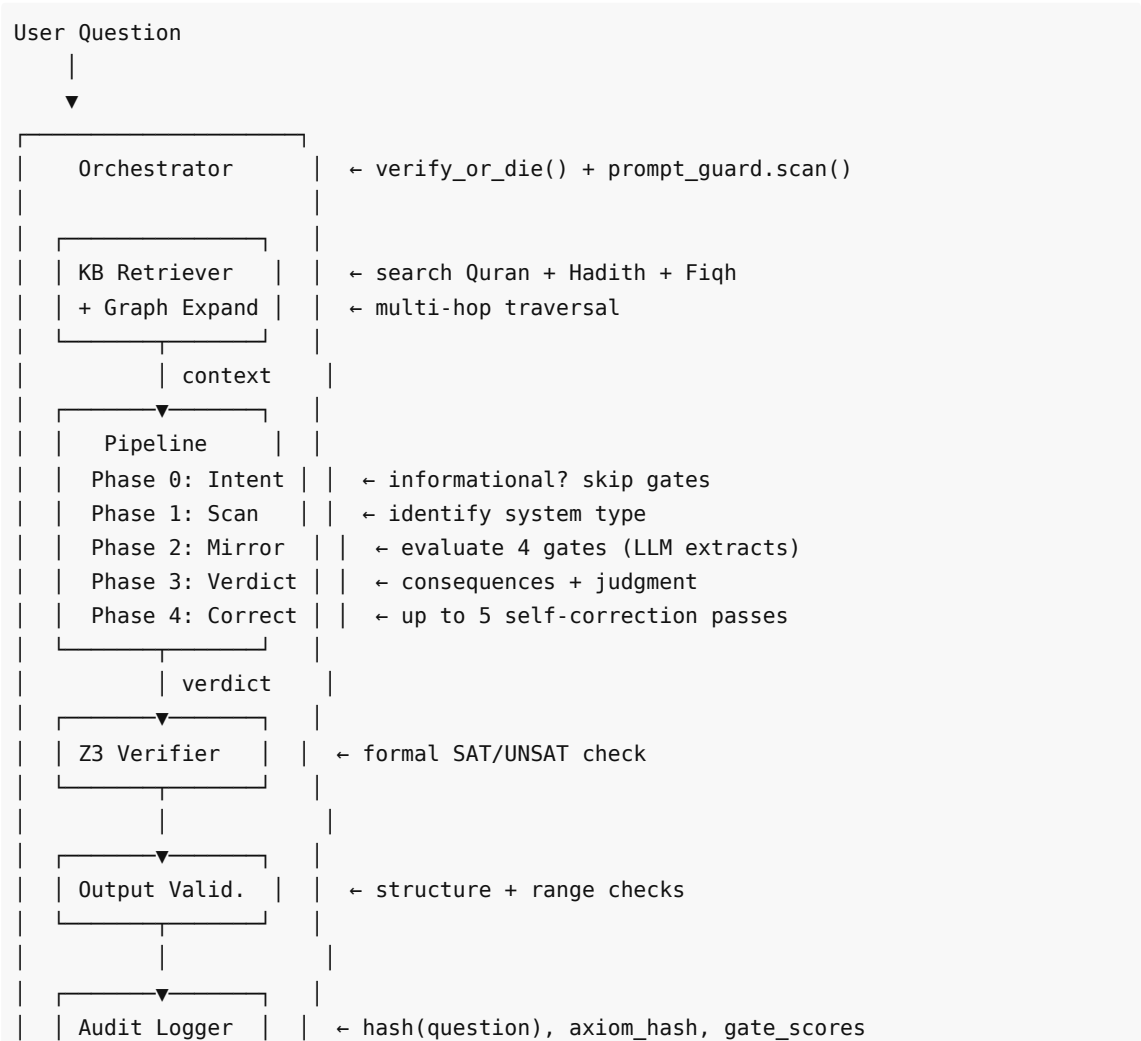
- 10. Log to audit trail (AuditLogger)
- 11. Return EvaluationResult

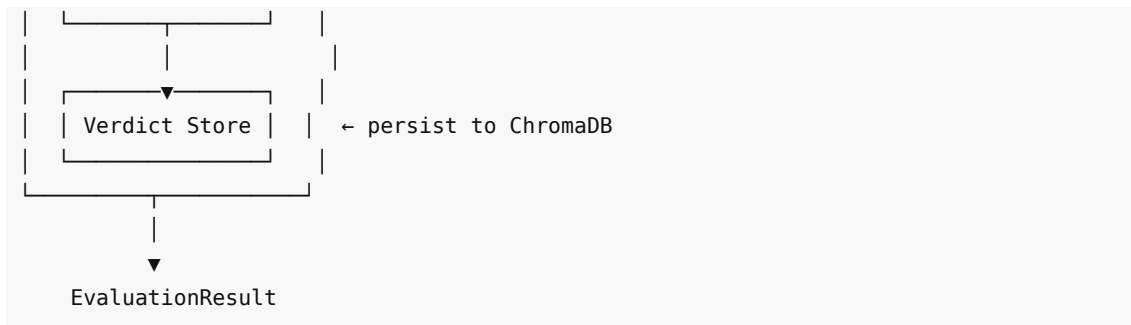
EvaluationResult

```
@dataclass
class EvaluationResult:
    response_text: str                # Natural language for user
    verdict: Verdict                  # Full verdict data
    dual_verdict: Optional[Verdict]  # If assumptions detected
    sources: list                     # KB sources used
    z3_result: Optional[Verdict]      # Formal verification
    evaluation_id: str                # Unique ID
    processing_time_ms: float         # Total time
    model_used: str                   # LLM model name
```

Tests: tests/test_orchestrator.py — 22 tests

13. End-to-End Data Flow





14. Test Coverage Summary

Engine Tests (tests/)

Test File	Count	Module
test_reasoning_engine.py	39	Legacy engine
test_cot.py	18	Chain of Thought v1
test_gate_source_integrity.py	12	Gate 1
test_gate_structural_consistency.py	8	Gate 2
test_gate_mediation_zeroing.py	8	Gate 3
test_gate_origin_aware.py	6	Gate 4
test_chain_executor.py	6	Chain execution
test_deterministic_scorer.py	9	Deterministic scoring
test_z3_axioms.py	19	Z3 axiom encoding
test_symbolic_verifier.py	14	Symbolic verification
test_predicate_extractor.py	12	Predicate extraction
test_embeddings.py	17	Embedding models
test_quran_collection.py	14	Quran collection
test_hadith_collection.py	13	Hadith collection
test_fiqh_collection.py	18	Fiqh collection
test_retriever.py	15	Unified retriever
test_graph_store.py	30	Graph store
test_graph_integration.py	12	Graph integration
test_knowledge_linker.py	12	Knowledge linker
test_kb_integration.py	6	KB end-to-end

test_orchestrator.py	22	Orchestrator
test_engine_integration.py	9	Engine integration
test_end_to_end.py	8	E2E pipeline
test_performance.py	6	Performance benchmarks
test_verdict_store.py	30	Verdict storage
test_feedback_store.py	15	Feedback storage
test_human_review.py	25	Human review
test_integrity_verifier.py	13	Integrity verification
test_prompt_guard.py	19	Prompt injection
test_output_validator.py	14	Output validation
test_adapter_sandbox.py	8	Adapter sandbox
test_audit_logger.py	9	Audit logging
test_security.py	7	Security integration
test_auth.py	13	Authentication
test_rate_limiter.py	17	Rate limiting
test_api_evaluate.py	9	API evaluate
test_api_verdicts.py	10	API verdicts
test_api_health.py	4	API health
test_config.py	15	Configuration
test_llm_layer.py	19	LLM provider
Total	560	

Skill Tests

Test File	Count	Module
furqan-raas/tests/test_mcp_server.py	31	RaaS MCP server
furqan-memory/tests/test_mcp_memory.py	16	Memory MCP server
furqan-memory/tests/test_memory_manager.py	14	Memory manager
furqan-memory/tests/test_sqlite_store.py	18	SQLite storage
furqan-memory/tests/test_vector_search.py	8	Vector search
Total	87	

Grand Total: 647 tests

Al-Furqan Engine Documentation — Sprints 3-5 — March 21, 2026 Variiance R&D — The Criterion Project